PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

Table of Contents

Email: umhackathon@um.edu.my

Project Name: Automatic Scholarship Application Agent (AI-Powered)

Version: 1.0

1. Project Overview

- **Problem Statement:** Students face fragmented, manual, and confusing workflows when applying for scholarships. They must repeatedly rewrite personal information, manually check eligibility across long documents, and rely on unreliable feedback for essays. Many waste time applying for scholarships they do not qualify for or miss opportunities entirely due to lack of awareness.
- **Target Domain:** Automating and optimizing scholarship application workflows using AI, focusing on eligibility analysis, essay evaluation, content reshaping, and intelligent opportunity matching.
- **Proposed Solution Summary:** An AI-powered scholarship application agent that centralizes student data, evaluates essays, checks eligibility, matches scholarships, reshapes content, tracks readiness, and manages applications. The system relies on AI (Gemini AI) as its core engine to analyze, evaluate, and guide users rather than generate content blindly.

2. Background & Business Objective

- **Background of the Problem:** Students currently navigate scholarship applications manually rewriting the same personal data repeatedly, reading long eligibility documents manually, receiving unreliable feedback from peers, missing smaller scholarship opportunities and struggling with structuring and adapting essays. This process is inefficient, error-prone, and mentally exhausting.

Email: umhackathon@um.edu.my

- **Importance of Solving This Issue:** The importance of solving this issue is that it prevents wasted time on ineligible applications, improves essay quality using structured evaluation, surfaces hidden scholarship opportunities, reduces repetitive manual work and provides objective, criteria-based feedback.
- **Strategic Fit / Impact:** It aligns with AI-first automation systems, demonstrates real-world AI dependency (not optional AI), positions the product as an intelligent decision-support system and enables scalable assistance for thousands of students.

3. Product Purpose

3.1. Main Goal of the System: To provide an AI-powered application assistant that evaluates, guides, and optimizes student scholarship applications rather than replacing human input.

3.2. Intended Users (Target Audience):

- Students applying for scholarships
- Pre-university students
- University students
- Scholarship applicants in Malaysia
- Students unfamiliar with application processes

4. System Functionalities

4.1. Description: The system operates as a centralized backend engine that stores student profiles, essays, and documents, stores scholarship and university requirements, Uses AI to evaluate, compare, and guide, tracks applications, readiness, and deadlines and also powers a full student dashboard. The backend consists of three main layers, the Core App Layer to handle auth, profile creation, essays, documents and applications validation and storage. The AI Intelligence Layer for scoring, eligibility, matching and reshaping and the Tracking Layer for dashboards, tracking deadlines, readiness and notifications.

4.2. Key Functionalities:

- **Core Backend Services**
  - Authentication Service
  - Profile Management
  - Scholarship & University Database
  - Essay Versioning System
  - Document Management
  - Application Tracking

- **AI-Powered Functionalities**

Essay Scoring & Feedback

- Provides overall score (e.g. 6.2/10)
- Breaks down evaluation criteria:
  - clarity of goals
  - specificity of achievements
  - alignment with scholarship values
  - grammar/formality
  - structure/flow
  - word efficiency
- Highlights weak sections and exact lines
- Suggests improvements

Eligibility Checker

- Compares:
  - Citizenship
  - CGPA
  - financial background
  - course requirements
- Returns:
  - eligible / not eligible
  - failed criteria
  - warnings
- Prevents wasted applications

Smart Scholarship Matching

- Ranks scholarships by fit
- Returns:
  - fit score
  - match reasoning
  - Risks
- Surfaces hidden opportunities

Content Reshaper

- Rewrites existing content for:
  - different questions
  - word limits
- Preserves student voice
- Avoids generating fake content

4.3. AI Model & Prompt Design

4.3.1. Model Selection: The product uses **Gemini 2.0 Flash** by Google DeepMind, chosen for its combination of low cost, fast response times, and strong instruction-following — all critical for a product that runs multiple AI calls in a single user session. Its ability to consistently return structured JSON output handles the complex nested schemas the system requires, its multilingual competence suits the Malaysian student context where essays may mix English and Malay, and its long context window comfortably fits the combined scholarship criteria, profile data, and essay text sent in each prompt.

4.3.2. Prompting Strategy: Our strategy for prompting uses multi-step agentic prompting, structured evaluation prompts and context-aware input (profile + essay + requirements). We chose these strategies as it ensures accurate scoring, reduces hallucination and enables consistent outputs.

4.3.3. Context & Input Handling: Our system accepts essays, profile data and scholarship requirements and handles large inputs via chunking and truncation. For inputs that are unstructured, the AI would automatically make it become a structured prompt. Gemini 2.0 Flash accepts up to 1,048,576 tokens (roughly 750,000 words) as input and outputs up to 8,192 tokens,

so for our use case there is essentially zero risk of hitting any limit. All five of our AI features, essay scoring, eligibility checking, scholarship matching, content reshaping, and autofill, deal with short inputs like essays and profile data that are tiny compared to that limit. Our backend currently sends everything in one shot with no chunking or truncation, which is perfectly fine. The only feature worth keeping an eye on is scholarship matching if the database grows to hundreds of entries, but our code already handles that by trimming each scholarship down to just the key fields before sending it to Gemini, so we are covered there too. If it has excess input, the input would be summarized or partially processed.

4.3.4. Fallback & Failure Behavior: When the model returns a bad, unusable, or hallucinated response, the system automatically retries the API call up to 2 more times with short delays before giving up. Each response is also validated before being accepted — for example, essay scores must fall within 0–10 and required fields must be present — so hallucinated or out-of-range values are caught and trigger another retry rather than being saved. If all attempts fail, the route returns a `502` error with a clear message to the frontend and nothing is written to the database, ensuring no corrupted data is ever stored. There is currently no formal human escalation path, but all failures are logged and surfaced clearly enough to support one in the future.

5. User Stories & Use Cases

- **User Stories:**
    1. "As a student, I want to reuse my content without rewriting everything."
    2. "As a student, I want to discover scholarships I didn't know existed."

- **Use Cases (Main Interactions):**

  Eligibility Check Flow

  - User selects scholarship
  - AI compares profile with requirements
  - Returns eligibility result

  Essay Evaluation Flow

  - User uploads essay

- Backend sends to AI
- AI returns structured feedback

Scholarship Matching Flow

- AI analyzes profile
- Returns ranked scholarships

Content Reshaping Flow

- User provides existing content
- AI adapts to new prompt

6. Features Included (Scope Definition)

- Essay scoring and feedback system
- Eligibility checking system
- Smart scholarship matching
- Content reshaping system
- Profile and document storage
- Application tracking system
- Dashboard with readiness tracking
- Deadline monitoring
- Notification system

7. Features Not Included (Scope Control)

- AI writing essays from scratch
- Interview coaching (full simulation)
- Multi-user collaboration
- Production-level deployment features
- Advanced frontend UI systems

8. Assumptions & Constraints

- **LLM Cost Constraint:** An average user session costs approximately 4,500–5,000 tokens. The key cost-control decision is the deliberate slimming of scholarship data before sending

it to the model in ‘match_scholarships’, reducing payload significantly per scholarship. Stored evaluation history also prevents redundant re-calls to the model.

- **Technical Constraints:** Key limitations include no database migration tooling, restricted file upload formats, no admin API for managing scholarship data, a non-functional ‘doc_expiry’ notification due to a missing model field, and no token refresh or password reset in the auth system.

- **Performance Constraints:** All AI calls are synchronous and blocking inside the Flask request cycle with no task queue, meaning concurrent users will cause timeouts. The matching feature sends the entire scholarship database in one prompt with no batching. There is also no caching layer anywhere — every request hits the database fresh.

- **User Input:** Every AI feature requires an explicit user-triggered API call — nothing runs automatically. All AI output is saved and served without any human approval or confidence threshold, meaning hallucinated results reach users directly. Prompts are hardcoded with no versioning, and autofill propagates profile data into forms without any external validation.

9. Risks & Questions Throughout Development

Essay Evaluation Accuracy

- How consistent are AI scores?

Eligibility Misinterpretation

- What if AI misreads criteria?

Scholarship Data Completeness

- Are all scholarships included?

User Trust

- Will users trust AI feedback?

Hallucination Risk

- How to prevent incorrect suggestions?

Over-Reliance on AI

- What happens if AI fails?

Email: umhackathon@um.edu.my